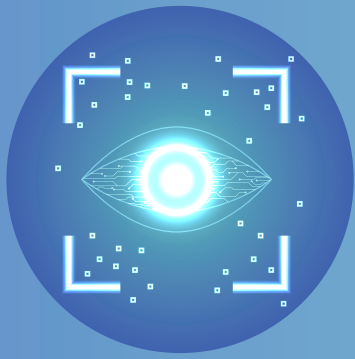




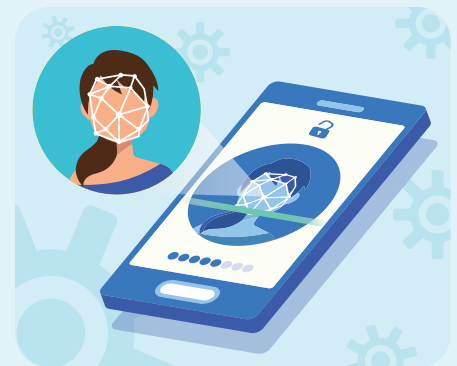
# Computer Vision

# 08

In the modern era of technology, machines are no longer just tools for processing numbers or executing commands; they can also 'see' and interpret their surroundings. With artificial intelligence, they can analyse images and videos, understand patterns, and make decisions based on visual data.



Consider facial recognition systems on smartphones. When users attempt to unlock a phone with their faces, the system scans their facial features, compares them to stored data, and verifies if they match the authorised user. This ability to interpret images and videos in real-time is made possible through a field of Artificial Intelligence called **Computer Vision**.



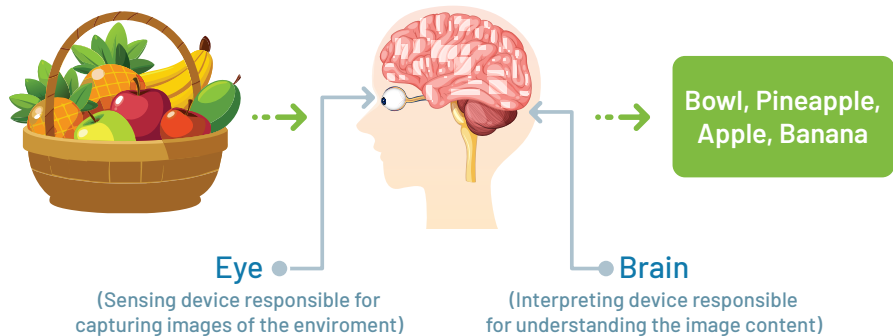
## Understanding Computer Vision

Computer Vision (CV) is a field of Artificial Intelligence (AI) that enables machines to interpret and understand information from images or videos. It allows computers to analyse, recognise, and extract meaningful data from the visual world.

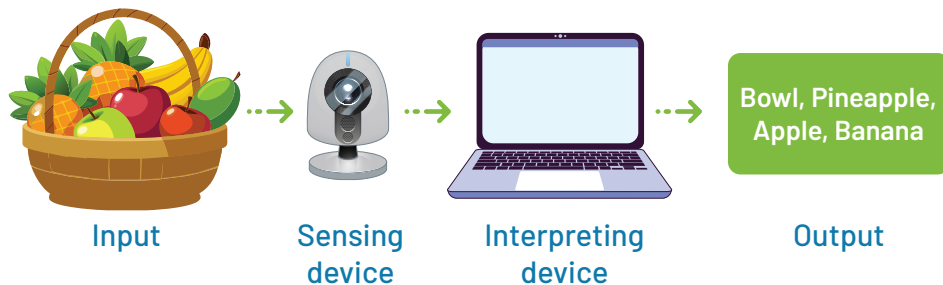


Computer Vision works in a way similar to the human brain. Both capture visual data—computers through cameras and humans through eyes—and process it to recognise objects. Computers analyse details such as shapes and colours, just as the brain interprets these features to understand the surroundings and make decisions.

### Human Vision System



### Computer Vision System



## The Working of Computer Vision

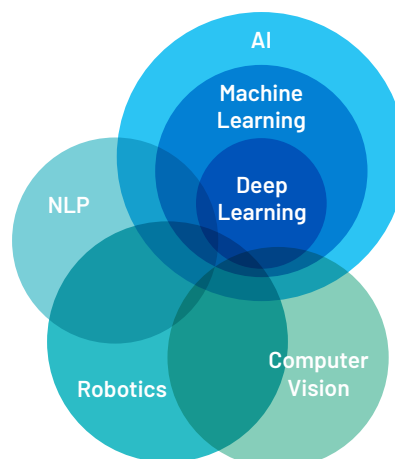


## Example

- Consider the example of a self-driving car. It relies on Computer Vision to navigate safely. First, its cameras capture images of the road and surroundings (Image Acquisition).
- Next, the system enhances these images by reducing noise and correcting distortions (Preprocessing). It then segments the images into different regions, allowing the system to focus on key areas such as lanes, pedestrians, and traffic signs (Segmentation).
- The system identifies key details such as shapes, colours, and edges (Feature Extraction). It compares them with stored data to recognise objects like other vehicles, pedestrians, and traffic signs (Pattern Recognition and Classification).
- Finally, based on this analysis, the car makes decisions such as stopping, turning, or adjusting its speed (Interpretation and Decision Making), ensuring safe navigation.



Computer Vision is closely connected to Machine Learning and Deep Learning, which enable systems to recognise patterns and make accurate predictions from visual data. It also integrates with Natural Language Processing (NLP) for tasks such as image captioning and collaborates with Robotics to support autonomous navigation.



## Note:

Image captioning is an AI technique that combines Computer Vision and Natural Language Processing to generate descriptive text for images.

## Applications of Computer Vision

Computer Vision is widely used across various sectors. Here are some key applications:

- **Facial Recognition:** In facial recognition systems, Computer Vision is used to identify and verify people by analysing facial features, which is helpful for security systems and unlocking smartphones.
- **Robotics:** Computer vision allows robots to understand their surroundings, recognise objects, and navigate through environments, which is crucial for automation and manufacturing.
- **Surveillance and Security:** Computer vision is used in video surveillance to monitor areas, detect unusual activities, and improve public safety.
- **Document Analysis:** Computer vision reads and interprets text from printed or handwritten documents, helping with data entry, digitisation, and archiving.

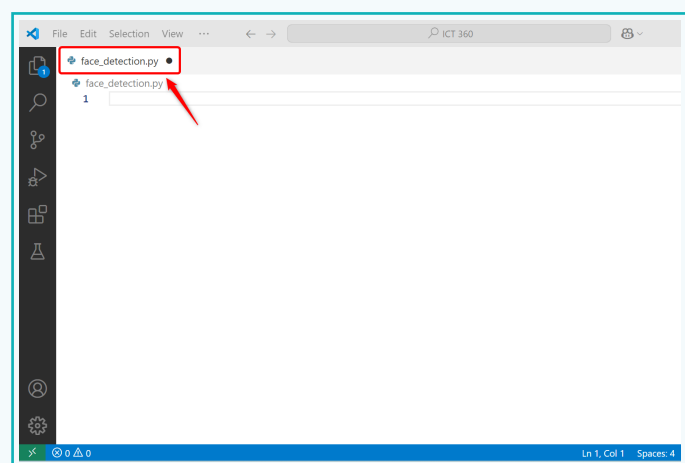
Computer vision has many more applications in almost every field, including medicine, education, and everyday life, enhancing efficiency and automation across various industries. Let us understand its implementation through a technical activity.

## Activity

Create a simple Face Detection program using Computer Vision in Python.

### Part 1 - Initial Setup

- 1 Open the VS Code editor, click the 'File' menu, and select 'Open Folder'. Navigate to the newly created folder and click on 'Select Folder'.
  - Once the folder appears in the sidebar, create a new file within it and name it "face\_detection.py".



## Activity

### Part 2 - Install Required Libraries

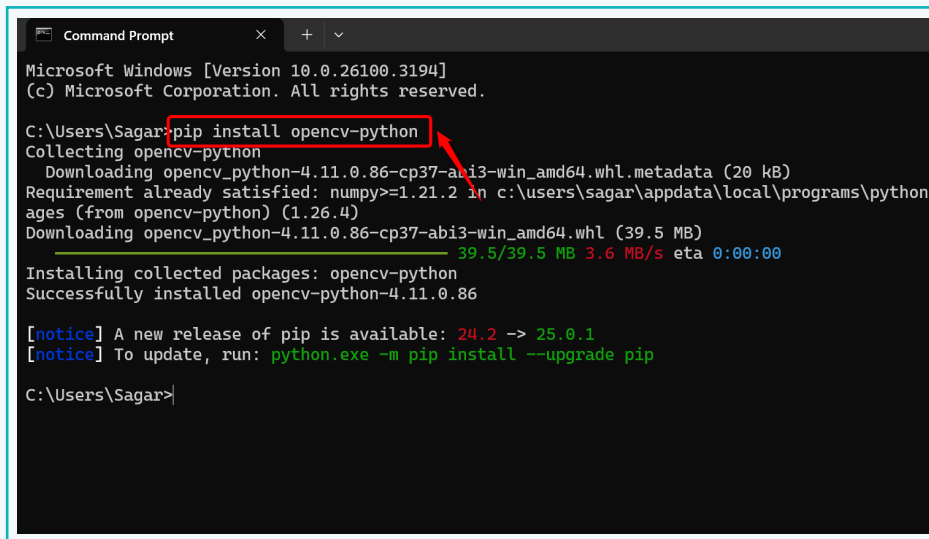
In order to implement face detection, we need to install the OpenCV library. It provides various tools and pre-trained models for detecting objects, including faces.

**Pre-Trained Model:** A pre-trained model is a machine learning model already trained on a large dataset for a specific task. In this case, we will use OpenCV's pre-trained 'Haar cascade' model for face detection.

**Haar Cascade Model:** A pre-trained machine learning model used for detecting objects like faces in images or videos. It works by identifying patterns of light and dark areas using Haar-like features.

- 1 Open the 'Command Prompt' and install OpenCV using the following command:

*pip install opencv-python*



```
Microsoft Windows [Version 10.0.26100.3194]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Sagar>pip install opencv-python
Collecting opencv-python
  Downloading opencv_python-4.11.0.86-cp37-abi3-win_amd64.whl.metadata (20 kB)
Requirement already satisfied: numpy>=1.21.2 in c:\users\sagar\appdata\local\programs\python\python39\lib\site-packages (from opencv-python) (1.26.4)
Downloading opencv_python-4.11.0.86-cp37-abi3-win_amd64.whl (39.5 MB)
39.5/39.5 MB 3.6 MB/s eta 0:00:00
Installing collected packages: opencv-python
Successfully installed opencv-python-4.11.0.86

[notice] A new release of pip is available: 24.2 -> 25.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\Sagar>
```

Once the library is successfully installed, we will load the model into our program.

### Part 3 - Load the Pre-trained Face Detection Model

- 1 Open the 'face\_detection.py' file and import the OpenCV library using the *import cv2* statement.

Now, load the 'Haar Cascade Classifier' model to detect frontal faces in images.



```
face_detection.py
1 import cv2
2
3
```

## Activity

We can import the Haar Cascade Classifier from 'cv2' using `cv2.CascadeClassifier()`. OpenCV provides default paths to pre-trained models through `cv2.data.harcascades`, allowing us to load the frontal face detection model by appending the XML file name to this path. This model is included in OpenCV by default and requires no additional downloads.

- 2 First, create a variable 'face\_cascade' to store the face detection model and use `cv2.CascadeClassifier()` to load the pre-trained face detection model.

```
1 import cv2
2
3 # Load the pre-trained Haar cascade for face detection
4 face_cascade = cv2.CascadeClassifier()
5
6
```

- 3 Inside `cv2.CascadeClassifier()`, specify the path to the Haar cascade file using `cv2.data.harcascades`. This provides the default path to pre-trained cascade files in OpenCV.

```
1 import cv2
2
3 # Load the pre-trained Haar cascade for face detection
4 face_cascade = cv2.CascadeClassifier(cv2.data.harcascades)
5
6
```

- 4 Append the file 'haarcascade\_frontalface\_default.xml' to load the frontal face detection model. This model is built into cv2 by default and does not require additional files.

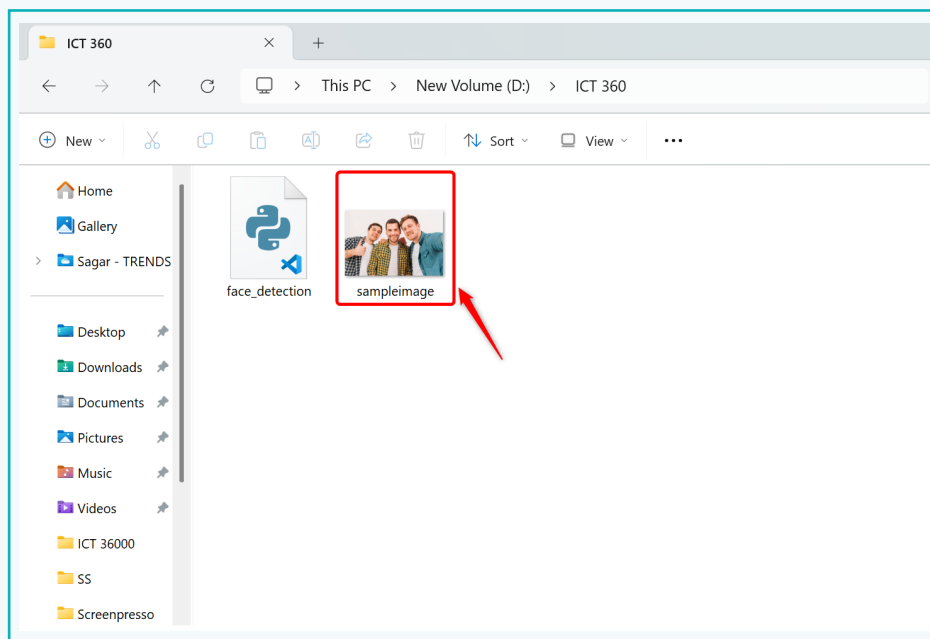
```
1 import cv2
2
3 # Load the pre-trained Haar cascade for face detection
4 face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
5
6
7
```

## Activity

### Part 4 - Load and Process the Image

Next, we must load an image and prepare it for face detection.

- 1 Select the sample image from the assets or download any image with a face and store it in the same folder where the 'face\_detection.py' file is saved.



- 2 First, create a variable 'image' to store the loaded image.
- 3 Use `cv2.imread()` to read the image from the specified file path.

```
4 face_cascade = cv2.CascadeClassifier(  
5  
6 # Load the image  
7 image = cv2.imread()  
8
```

## Activity

- 4 Pass the image file path inside `cv2.imread()` and use a *raw string* (`r""`) before the file path to ensure backslashes are correctly interpreted.

### Note:

You can get the image file path by right-clicking the sample image in your folder and selecting 'Copy Path'.

```
4 face_cascade = cv2.CascadeClassifier(cv2.data.haarcas
5
6 # Load the image
7 image = cv2.imread(r"D:\ICT 360\sampleimage.jpg")
8
```

Before detecting faces, we convert the image into grayscale because face detection algorithms work better with grayscale images.

## Grayscale Images

Grayscale images contain only shades of grey, unlike normal colour images, which have multiple colour channels (RGB). Each pixel in a grayscale image has a single intensity value, making it easier for face detection algorithms to process.

Normal Image

Grayscale Image



Image Credits: ResearchGate

Grayscale conversion is commonly used in most applications because it increases processing speed and detection accuracy. Colour images are generally avoided, since they add unnecessary complexity and can be influenced by lighting and colour variations, making face detection less reliable.

## Activity

- 5 First, create a variable 'gray\_image' to store the converted grayscale image and use `cv2.cvtColor()` to change the image's colour format.

```
7 image = cv2.imread(r"D:\ICT 360\sampleimage.jpg")
8
9 # Convert the image to grayscale
10 gray_image = cv2.cvtColor()
11
12
```

- 6 Pass the original image variable, 'image', as the first argument inside `cv2.cvtColor()` and use `cv2.COLOR_BGR2GRAY` to specify the conversion from BGR (Blue-Green-Red) to Grayscale.

```
7 image = cv2.imread(r"c:\Users\Sagar\Desktop\Programs\sample
8
9 # Convert the image to grayscale
10 gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
11
12
```

### Part 5 - Detect Faces in the Image

Now, we will detect faces using the `detectMultiScale()` function.

**`detectMultiScale()`:** It detects objects (here faces) in an image. It scans images at different scales and identifies objects, returning their coordinates as rectangular bounding boxes.

- 1 First, create a variable 'faces' to store the detected face coordinates and use `face_cascade.detectMultiScale()` to perform face detection on the 'gray\_image'.

```
10 gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
11
12 # Detect faces in the image
13 faces = face_cascade.detectMultiScale()
14
15
```

## Activity

- Pass 'gray\_image' as the first argument to detect faces in the preprocessed image and set 'scaleFactor=1.1' to slightly reduce the image size at each scale, improving detection accuracy.

```
10 gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
11
12 # Detect faces in the image
13 faces = face_cascade.detectMultiScale(gray_image, scaleFactor=1.1)
14
15
```

### Note:

The 'scaleFactor=1.1' reduces the image size by 10% per scale, allowing the algorithm to detect faces of different sizes. A lower value increases accuracy but slows detection, while a higher value speeds up detection but may miss smaller faces.

- Set 'minNeighbors=5' and 'minSize=(30, 30)' as shown.

```
9 # Convert the image to grayscale
10 gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
11
12 # Detect faces in the image
13 faces = face_cascade.detectMultiScale(gray_image, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))
14
15
```

Here,

**minNeighbors=5** ensures a face is detected only if at least five overlapping detections occur in the same area. A higher value makes detection more selective but may miss some faces, while a lower value increases detection but may include non-face objects.

**minSize=(30, 30)** ensures the detector ignores tiny objects and detects faces that are 30x30 pixels or larger, improving accuracy by focusing on meaningful face sizes.

## Activity

### Part 6 - Mark the Detected Faces

Face detection applications often highlight detected faces with rectangular boxes. To visualise these detections, we draw rectangles around them using OpenCV's `rectangle()`.

**`rectangle()`:** It is used for highlighting objects, taking parameters like the image, starting and ending coordinates, colour, and thickness.

#### Syntax:

```
cv2.rectangle(image, start_point, end_point, color, thickness)
```

Where,

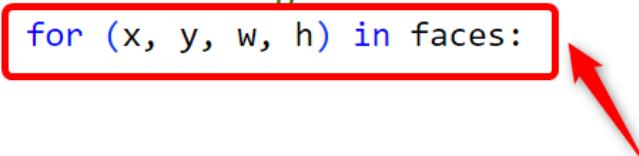
- *image*: The image on which the rectangle is drawn.
- *start\_point*: The top-left corner of the rectangle (x, y).
- *end\_point*: The bottom-right corner of the rectangle (x, y).
- *color*: The colour of the rectangle (B, G, R).
- *thickness*: The thickness of the rectangle's border (-1 fills the rectangle).

Now, the 'faces' variable stores a list of detected faces, where each face is represented as a tuple (x, y, w, h).

- 'x', 'y' represents the top-left coordinates of the detected face.
- 'w', 'h' represents the width and height of the face.

1 Loop through each detected face using a *for* loop.

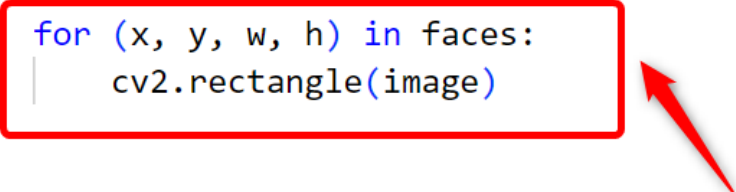
```
13 faces = face_cascade.detectMultiScale(gray_im
14
15 # Draw rectangles around the detected faces
16 for (x, y, w, h) in faces:
17
18
```



## Activity

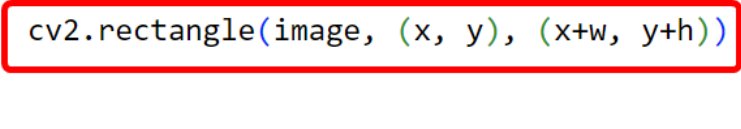
- In the *for* loop, use *cv2.rectangle()* to draw a rectangle around each detected face on the 'image'.

```
14
15 # Draw rectangles around the detected faces
16 for (x, y, w, h) in faces:
17     cv2.rectangle(image)
18
19
```



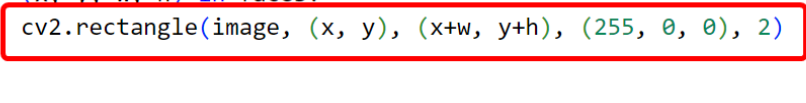
- Pass (x, y) as the starting point (top-left corner) of the rectangle and pass (x + w, y + h) as the ending point (bottom-right corner) of the rectangle.

```
14
15 # Draw rectangles around the detected faces
16 for (x, y, w, h) in faces:
17     cv2.rectangle(image, (x, y), (x+w, y+h))
18
19
```



- Set (255, 0, 0) as the rectangle colour (blue in BGR format) and set 2 as the thickness of the rectangle border.

```
14
15 # Draw rectangles around the detected faces
16 for (x, y, w, h) in faces:
17     cv2.rectangle(image, (x, y), (x+w, y+h), (255, 0, 0), 2)
18
19
```



## Activity

Now, we must write the code to display the processed image with the detected faces. For this, we need to create a window to show the image. Then, the window should pause until we press any key. Once a key is pressed, the window should close automatically.

2 Use `cv2.imshow()` to open a window named 'Face Detection' and display the processed 'image'.

- Use `cv2.waitKey(0)` to pause execution until a key is pressed. The '0' means it will wait indefinitely.
- Use `cv2.destroyAllWindows()` to close all OpenCV windows after a key is pressed.

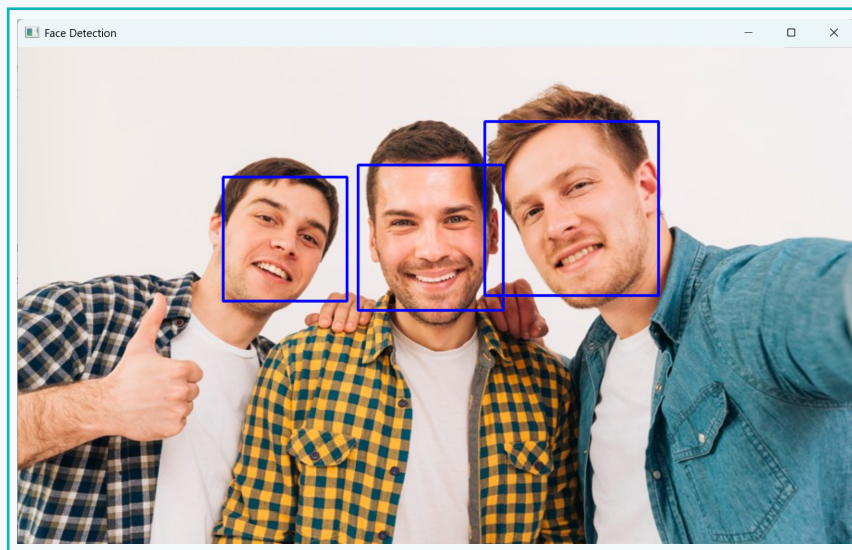
```
17     cv2.rectangle(image, (x, y), (x+w, y+h),  
18  
19     # Display the result  
20     cv2.imshow('Face Detection', image)  
21     cv2.waitKey(0)  
22     cv2.destroyAllWindows()  
23  
24
```

## Part 7 - Test the Face Detection

Try executing the program. It will detect faces in the given image and highlight them with blue rectangles. The detected faces will be displayed in a window.

You can test the program with different images by changing the image file's path in the `cv2.imread()` function.

## Output -



In this lesson, we discovered how computers can see and understand the visual world using Computer Vision. We built a face detection system using a pre-trained model, processed images and finally identified the faces, demonstrating how machines recognise objects in images.

## Exercise

### Fill in the Blanks.

- 1 \_\_\_\_\_ is a field of Artificial Intelligence (AI) that enables machines to interpret and understand information from images or videos.
- 2 A \_\_\_\_\_ model is a machine learning model already trained on a large dataset for a specific task.
- 3 `cv2.COLOR_BGR2GRAY` is used to specify the conversion from BGR (Blue-Green-Red) to \_\_\_\_\_.

### State if the statements are True or False.

- 4 Computer Vision is closely connected to Machine Learning and Deep Learning, which enable systems to recognise patterns and make accurate predictions from visual data.



\_\_\_\_\_

- 5 Computer vision is used in video surveillance to monitor areas, detect unusual activities, and improve public safety.



\_\_\_\_\_

- 6 Grayscale images contain all colours except grey.



\_\_\_\_\_

## Tech Flash

- |                           |   |                                                                                                                                    |
|---------------------------|---|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Computer Vision</b>    | : | It is a field of Artificial Intelligence (AI) that enables machines to interpret and understand information from images or videos. |
| <b>Image Captioning</b>   | : | It is an AI technique that combines Computer Vision and Natural Language Processing to generate descriptive text for images.       |
| <b>Pre-Trained Model</b>  | : | It is a machine learning model already trained on a large dataset for a specific task.                                             |
| <b>Haar Cascade Model</b> | : | A pre-trained machine learning model used for detecting objects like faces in images or videos.                                    |